

closure_evidence_20260925

BOB-236 closure evidence — qbittorrent-proxy ignores SIGTERM / SIGKILLed after 10s grace

Date (UTC): 2026-09-25 **Item:** BOB-236 (Bug) — “qbittorrent-proxy ignores SIGTERM and is SIGKILLed after the 10 s grace period on every restart” **Method:** §11.4.102 systematic-debugging (Iron Law: root cause before fix); §11.4.115 RED-baseline-on-the-broken-artifact; §11.4.43 TDD-fix workflow.

This document is machine-verifiable §11.4.5/§11.4.69 evidence, not a narrative claim. Every timestamp/log line below was captured in this session against the real, live qbittorrent-proxy container and the real unit-test suite. No commit/push and no workable-items close was performed — that is the conductor’s job per instructions.

1. Root cause (proven, not guessed)

PID 1 inside the container is python3 /config/download-proxy/src/main.py directly (confirmed via podman top below — start-proxy.sh ends with exec python3 "\$SRC_DIR/main.py", so the shell-wrapper-eats-signal candidate is **ruled out**).

main.py::main() installs a SIGTERM handler that only does _shutdown_event.set() (a threading.Event). The main loop wakes on it and runs:

```
logger.info("Shutting down...")
proxy_thread.join(timeout=5)
```

```
fastapi_thread.join(timeout=5)
logger.info("Shutdown complete")
```

Neither background thread was ever wired to actually stop:

1. **proxy_thread** (`download_proxy.run_server()`, port 7186) — `ThreadingHTTPServer.serve_forever()` only called `httpd.shutdown()` on `KeyboardInterrupt`. CPython never raises `KeyboardInterrupt` on a non-main thread, and this function always runs on a background daemon thread. So `SIGTERM` never reached this server at all.
2. **fastapi_thread** (`uvicorn.Server.serve()`, port 7187) — uvicorn's own `Server.capture_signals()` (`uvicorn/server.py:323-327`) explicitly does:

```
if threading.current_thread() is not threading.main_thread():
    # Signals can only be listened to from the main thread.
    yield
    return
```

Since `asyncio.run(_serve_with_heartbeat())` always runs on the background `fastapi_thread`, uvicorn never installs its own `SIGTERM` handler and nothing else in the codebase ever set `server.should_exit = True`. `await server.serve()` therefore ran forever regardless of the process-level shutdown signal.

Consequence: both `.join(timeout=5)` calls were *guaranteed* to fully time out on every `SIGTERM` (5s + 5s = 10.0s), landing shutdown right at — and, with logging/scheduling overhead, consistently just past — the container's default 10s `StopTimeout` (`docker-compose.yml` sets no `stop_grace_period`; podman confirms the container's configured default below), forcing a `SIGKILL` on effectively every restart. This is not a shell-PID-1 issue, not a blocked event loop, and not B0B-137 stall-related — it is two background service loops that were structurally never told to stop.

2. RED baseline — defect reproduced live on the pre-fix artifact

Container's configured stop timeout (confirms the “10 second grace period” in the ticket):

```
$ podman inspect qbittorrent-proxy --format
'{{.Config.StopTimeout}}'
10
```

PID 1 confirmed (rules out the shell-wrapper candidate):

```
$ podman top qbittorrent-proxy -eo pid,ppid,comm,args
      PID      PPID COMMAND      COMMAND
      1         0 python3      python3 /config/download-proxy/
src/main.py
      372        1 python      [python] <defunct>
      ...
```

RED #1 — extended grace (30s), proves the process *eventually* exits cleanly but far too slowly

```
2026-09-25T08:10:15Z
podman stop rc=0 elapsed=11.044408462s
```

Container log (timestamps):

```
2026-09-25T10:10:15.632929000+02:00 2026-09-25 08:10:15,632 - INFO
- Shutting down...
2026-09-25T10:10:25.633725000+02:00 2026-09-25 08:10:25,633 - INFO
- Shutdown complete
```

Delta = **10.000796s exactly** between “Shutting down...” and “Shutdown complete” — the two sequential `.join(timeout=5)` calls both fully timed out, confirming neither thread ever exits on its own.

RED #2 — default 10s grace, the exact production invocation (`start.sh/compose` use no `--time` override)

```
2026-09-25T08:13:00Z
podman stop rc=0 elapsed=10.171619266s container_exitcode=137
time="2026-09-25T10:13:10+02:00" level=warning msg="StopSignal
SIGTERM failed to stop container qbittorrent-proxy in 10 seconds,
resorting to SIGKILL"
```

`container_exitcode=137 = SIGKILL`. The logged warning is **verbatim** the symptom quoted in the BOB-236 description. This is the RED-baseline-on-the-broken-artifact required by §11.4.115.

RED #3 — unit-level reproduction of the specific mechanism (`plugins/download_proxy.py::run_server()`)

With the fix in `plugins/download_proxy.py` temporarily reverted in place (no git operations — `.git/index.lock` was held by a stale, non-live lock from an unrelated crashed process; touching git was out of scope for this task, so the revert was done via in-memory file edit and restored immediately after capturing evidence):

```
$ .venv/bin/python -m pytest tests/unit/
test_download_proxy_deep.py::TestRunServer::test_run_server_calls_l
-v
```

```
PytestUnhandledThreadExceptionWarning: Exception in thread
Thread-1 (run_server)
Traceback (most recent call last):
```

```
...
TypeError: run_server() takes 0 positional arguments but 1 was
given
...
E           AssertionError: Expected 'shutdown' to have been
called once. Called 0 times.
FAILED tests/unit/
test_download_proxy_deep.py::TestRunServer::test_run_server_calls_l
1 failed, 1 warning in 0.39s
```

Full log: this session's transcript (also saved to the scratchpad during the session as `bob236_red_unittest.log`).

3. The fix

`plugins/download_proxy.py`

`run_server()` now accepts an optional `shutdown_event`:
`threading.Event` | `None`. When supplied, a small daemon watcher thread blocks on `shutdown_event.wait()` and calls `httpd.shutdown()` the moment it fires — `socketserver.BaseServer.shutdown()` is documented as safe to call from a different thread while `serve_forever()` runs elsewhere, and makes it return within one polling interval (default 0.5s).

download-proxy/src/main.py

- `start_original_proxy()` now calls `run_server(_shutdown_event)` instead of `run_server()`.
- `start_fastapi_server()`'s `_serve_with_heartbeat()` now also starts an `_await_process_shutdown()` task that awaits `_shutdown_event.wait()` off-loop (`loop.run_in_executor`) and then sets `server.should_exit = True` — the exact flag uvicorn's own `on_tick()` polls every 0.1s (`uvicorn/server.py:233-262`).

No change was needed to `main()`'s join structure — the root cause was “never signaled,” not “joins are sequential”; fixing the signaling makes both joins return almost immediately.

Deployment: `plugins/download_proxy.py` is an “infra module” copied by `./install-plugin.sh --all` into `config/qBittorrent/nova3/engines/download_proxy.py` (the actual import path inside the container, per `ENGINES_DIR`), then `download-proxy/src/` is bind-mounted directly. Both steps were run this session (`./install-plugin.sh --all`, then `./start.sh --reload-python`) to make the fix live.

4. GREEN — fix confirmed live, same exact invocation as the RED baseline

Container restarted with the fix live and confirmed healthy:

```
$ podman ps --filter name=qbittorrent-proxy --format "{{.Status}}"
Up 31 seconds (healthy)
```

Same exact default-timeout invocation as RED #2:

```
2026-09-25T08:17:27Z
podman stop rc=0 elapsed=.588671048s container_exitcode=0
```

Container log (timestamps) — full application-level shutdown, including uvicorn's own graceful-lifespan shutdown now actually engaging:

```
2026-09-25T10:17:27.561119000+02:00 2026-09-25 08:17:27,560 - INFO
- Shutting down...
2026-09-25T10:17:27.651920000+02:00 INFO:      Shutting down
2026-09-25T10:17:27.752402000+02:00 INFO:      Waiting for
application shutdown.
2026-09-25T10:17:27.752983000+02:00 2026-09-25 08:17:27,752 - INFO
- Saved 0 scheduled searches
```

```

2026-09-25T10:17:27.753066000+02:00 2026-09-25 08:17:27,752 - INFO
- Scheduler stopped
2026-09-25T10:17:27.753210000+02:00 2026-09-25 08:17:27,753 - INFO
- Merge Service API stopped
2026-09-25T10:17:27.753410000+02:00 INFO:      Application shutdown
complete.
2026-09-25T10:17:27.753536000+02:00 INFO:      Finished server
process [1]
2026-09-25T10:17:27.884697000+02:00 2026-09-25 08:17:27,884 - INFO
- Shutdown complete

```

Delta “Shutting down...” → “Shutdown complete” = **0.323s** (was 10.000796s pre-fix). No SIGKILL warning. container_exitcode=0 (graceful exit), was 137 pre-fix.

Metric	Pre-fix (RED)	Post-fix (GREEN)
podman stop wall time (default 10s grace)	10.17s	0.59s
Container exit code	137 (SIGKILLed)	0 (graceful)
Runtime warning	StopSignal SIGTERM failed... resorting to SIGKILL	none
App-level shutdown log delta	10.0008s	0.323s

Unit-level GREEN (mechanism-level regression guard)

```

$ .venv/bin/python -m pytest tests/unit/
test_download_proxy_deep.py -v --import-mode=importlib
...
tests/unit/
test_download_proxy_deep.py::TestRunServer::test_run_server_calls_l
PASSED
...
32 passed in 2.02s

```

Standing integration regression guard (drives the REAL live container, no mocks)

```

$ .venv/bin/python -m pytest tests/integration/
test_bob236_sigterm_graceful_shutdown.py -v --import-
mode=importlib
tests/integration/
test_bob236_sigterm_graceful_shutdown.py::test_sigterm_stops_conta:

```

PASSED

1 passed in 33.68s

(This test does a real `podman/docker stop <container>` with the container's default configured timeout — no override — and asserts (a) exit code is never 137/SIGKILL and (b) wall time is comfortably below the 5s bound, well under the container's 10s grace period. The 33.68s total run time is fixture teardown re-waiting for the container to report healthy again, not the assertion itself.)

Full suite sanity (no regressions)

```
$ .venv/bin/python -m pytest tests/unit/test_main.py tests/unit/
test_download_proxy_deep.py --import-mode=importlib -q
63 passed, 4 warnings in 5.96s
```

(The 4 warnings are a pre-existing, unrelated `RuntimeWarning: coroutine ... was never awaited` in tests that fully mock `asyncio.run` — present before this change too, confirmed unaffected by re-inspection.)

Live service health after fix (both ports)

```
$ curl -sf http://localhost:7187/health
{"status":"healthy","service":"merge-search","version":"1.3.0"}
$ curl -sf -o /dev/null -w "%{http_code}\n" http://localhost:7186/
200
```

5. Files changed

- `plugins/download_proxy.py` — `run_server()` gains optional `shutdown_event` param + `watcher` thread; `import threading` added. (Also propagated to the deployed copy `config/qBittorrent/nova3/engines/download_proxy.py` via `./install-plugin.sh --all`, per repo convention — that destination file is generated/deployed, not hand-edited.)
- `download-proxy/src/main.py` — `start_original_proxy()` passes `_shutdown_event` through; `start_fastapi_server()`'s `_serve_with_heartbeat()` gains an `_await_process_shutdown()` task that sets `server.should_exit = True`.
- `tests/unit/test_download_proxy_deep.py` — new RED/GREEN regression test
`TestRunServer::test_run_server_calls_httpd_shutdown_when_shutdown_event_is_set` (+ `threading/time` imports).

- `tests/integration/test_bob236_sigterm_graceful_shutdown.py` — new standing §11.4.135 regression guard driving the real live container.

6. Not done (out of scope per instructions)

- No `git commit/git push` (conductor's job).
- No `constitution/scripts/workable-items/bin/workable-items` close invocation (left for the conductor after independent verification, per §11.4.240 `producer=verifier`).
- The stale `.git/index.lock` (dated 2026-09-23, no live holder found via `ps`) was left untouched — reaping it is not this task's scope and destructive `git-state` changes are the conductor's responsibility.